

# Volume 03 - Book 03.01 Enterprise Core

Version v3 draft

README.md

## Book 03.01 - Enterprise Core

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-README

### Purpose

Define the Enterprise Core blueprint package and its subsystem decomposition.

### Scope

Covers the Book 03.01 folder, its internal reading order, and its role as the first detailed Master Blueprint book.

### Responsibilities

- Define blueprint authority for the Enterprise Core area.
- Maintain alignment with the Volume 02 Enterprise Core roadmap phase.
- Give later specifications enough structure to become implementation-ready.

### Inputs

- Volume 00 Enterprise Constitution principles.
- Volume 01 Master Vision strategy.
- Volume 02 Master Roadmap phase definitions.

### Outputs

- Canonical blueprint structure for Enterprise Core.
- Traceable subsystem boundaries.
- Acceptance expectations for later specifications and implementation.

### Interfaces

- Volume 03 Master Blueprint index.
- Book 03.01 Enterprise Core subsystem documents.
- Future Volume 06 subsystem specifications.

### Events

- BlueprintChanged
- SubsystemBoundaryDefined
- AcceptanceRuleUpdated

## Storage

- Markdown source files in the architecture library.
- Generated PDF release artifacts.
- Release manifest checksums.

## Security

- No secrets, credentials, private keys, or operational tokens are stored in blueprint files.
- Security-sensitive subsystem behavior is described as architecture, not executable configuration.
- Access-control decisions are deferred to approved specifications.

## Metrics

- Document completeness ratio.
- Traceability coverage across roadmap and constitution sources.
- Release artifact generation success.

## Tests

- Verify required sections exist in every file.
- Verify generated PDF is readable.
- Verify v3 ZIP contains Markdown, PDFs, manifest, and release notes.

## Acceptance Criteria

- The document contains all required v3 sections.
- The document can guide downstream specifications without code changes.
- The document is included in generated release artifacts.

## Traceability

- MAL-V00-B005
- MAL-V00-B011
- MAL-V02-B003

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01 Index

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-INDEX

## Purpose

List the canonical Enterprise Core subsystem blueprint files.

## Scope

Covers Book 03.01 documents from overview through core roadmap.

## Responsibilities

- Define blueprint authority for the Enterprise Core area.
- Maintain alignment with the Volume 02 Enterprise Core roadmap phase.
- Give later specifications enough structure to become implementation-ready.

## Inputs

- Volume 00 Enterprise Constitution principles.
- Volume 01 Master Vision strategy.
- Volume 02 Master Roadmap phase definitions.

## Outputs

- Canonical blueprint structure for Enterprise Core.
- Traceable subsystem boundaries.
- Acceptance expectations for later specifications and implementation.

## Interfaces

- Volume 03 Master Blueprint index.
- Book 03.01 Enterprise Core subsystem documents.
- Future Volume 06 subsystem specifications.

## Events

- BlueprintChanged
- SubsystemBoundaryDefined
- AcceptanceRuleUpdated

## Storage

- Markdown source files in the architecture library.
- Generated PDF release artifacts.

- Release manifest checksums.

## Security

- No secrets, credentials, private keys, or operational tokens are stored in blueprint files.
- Security-sensitive subsystem behavior is described as architecture, not executable configuration.
- Access-control decisions are deferred to approved specifications.

## Metrics

- Document completeness ratio.
- Traceability coverage across roadmap and constitution sources.
- Release artifact generation success.

## Tests

- Verify required sections exist in every file.
- Verify generated PDF is readable.
- Verify v3 ZIP contains Markdown, PDFs, manifest, and release notes.

## Acceptance Criteria

- The document contains all required v3 sections.
- The document can guide downstream specifications without code changes.
- The document is included in generated release artifacts.

## Traceability

- MAL-V00-B005
- MAL-V00-B011
- MAL-V02-B003

## Book Files

1. book-03-01-01-enterprise-core-overview.md
2. book-03-01-02-kernel.md
3. book-03-01-03-lifecycle.md
4. book-03-01-04-runtime-manager.md
5. book-03-01-05-runtime-registry.md
6. book-03-01-06-event-bus.md
7. book-03-01-07-service-container.md
8. book-03-01-08-configuration.md
9. book-03-01-09-identity-access.md
10. book-03-01-10-state-manager.md
11. book-03-01-11-resource-manager.md
12. book-03-01-12-object-manager.md

- 13. book-03-01-13-health-manager.md
- 14. book-03-01-14-core-events.md
- 15. book-03-01-15-core-storage.md
- 16. book-03-01-16-core-security.md
- 17. book-03-01-17-core-observability.md
- 18. book-03-01-18-core-testing.md
- 19. book-03-01-19-core-acceptance.md
- 20. book-03-01-20-core-roadmap.md

Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.01 - Enterprise Core Overview

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-001

## Purpose

Define the Enterprise Core as the foundational execution substrate for Mariam AI Enterprise OS.

## Scope

Covers core boundaries, subsystem map, operational expectations, and how Enterprise Core supports all later platform layers.

## Responsibilities

- Define the kernel-centered core architecture.
- Separate enterprise primitives from domain modules.
- Establish subsystem ownership for lifecycle, runtime, identity, state, resources, objects, health, storage, security, observability, and testing.

## Inputs

- Enterprise roadmap phase from MAL-V02-B003.
- Constitutional enterprise, security, quality, and engineering principles.
- Master Vision enterprise objectives.

## Outputs

- Enterprise Core subsystem map.
- Blueprint boundaries for core services.
- Implementation-ready decomposition for future specifications.

## Interfaces

- Kernel API boundary.
- Runtime manager contract.
- Identity and access boundary.
- Core event and storage contracts.

## Events

- EnterpriseCoreInitialized
- EnterpriseCoreStarted
- EnterpriseCoreDegraded
- EnterpriseCoreStopped

## Storage

- Core configuration records.
- Subsystem registry records.
- Audit event references.
- Health snapshots.

## Security

- Apply least privilege across core services.
- Keep tenant and role context explicit.
- Audit protected core actions.

## Metrics

- Core startup time.
- Subsystem readiness coverage.
- Core error rate.
- Audit event completeness.

## Tests

- Blueprint section validation.
- Subsystem dependency review.
- PDF readability verification.

## Acceptance Criteria

- All Enterprise Core subsystems are named and scoped.
- Downstream specifications can reference stable subsystem boundaries.
- No domain feature bypasses the core governance layer.

## Traceability

- MAL-V00-B005
- MAL-V00-B012
- MAL-V01-B002
- MAL-V02-B003

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.02 - Kernel

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-002

## Purpose

Define the Mariam Kernel as the minimal trusted coordinator for core boot, service access, and platform invariants.

## Scope

Covers kernel responsibilities, allowed dependencies, startup responsibilities, core service exposure, and failure boundaries.

## Responsibilities

- Own the platform boot sequence.
- Expose stable access to core services.
- Enforce core invariants before subsystem startup.
- Prevent domain modules from coupling directly to infrastructure details.

## Inputs

- Boot configuration.
- Runtime registry definitions.
- Service container bindings.
- Identity and security policy handles.

## Outputs

- Kernel readiness state.
- Core service handles.
- Kernel lifecycle events.
- Boot diagnostics.

## Interfaces

- Kernel boot API.
- Service container interface.
- Runtime manager interface.
- Health manager interface.

## Events

- KernelBootRequested
- KernelBooted



- KernelInvariantFailed
- KernelShutdownRequested

## Storage

- Kernel boot log.
- Core invariant registry.
- Kernel state snapshot.

## Security

- Only trusted boot code can mutate kernel state.
- Kernel APIs must validate caller context.
- Boot diagnostics must not expose secrets.

## Metrics

- Boot duration.
- Invariant failure count.
- Kernel panic count.
- Service resolution latency.

## Tests

- Kernel boot unit tests.
- Invariant failure tests.
- Unauthorized service access tests.
- Shutdown sequence tests.

## Acceptance Criteria

- Kernel boots with required services available.
- Kernel refuses unsafe startup state.
- Kernel shutdown is orderly and observable.

## Traceability

- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B003

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.03 - Lifecycle

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-003

## Purpose

Define lifecycle states and transitions for Enterprise Core startup, operation, degradation, maintenance, and shutdown.

## Scope

Covers lifecycle state machines for the core platform and subsystem readiness.

## Responsibilities

- Define canonical lifecycle states.
- Coordinate subsystem startup and shutdown.
- Expose degradation and maintenance modes.
- Prevent invalid state transitions.

## Inputs

- Kernel state.
- Subsystem readiness reports.
- Runtime manager commands.
- Health manager signals.

## Outputs

- Lifecycle transition records.
- Readiness decisions.
- Maintenance mode state.
- Shutdown completion signals.

## Interfaces

- Kernel lifecycle hooks.
- Runtime manager lifecycle API.
- Event bus lifecycle topics.
- Health manager readiness API.

## Events

- LifecycleTransitionRequested
- LifecycleTransitionAccepted

- LifecycleTransitionRejected
- LifecycleStateChanged

## Storage

- Lifecycle transition journal.
- Readiness snapshots.
- Maintenance window records.

## Security

- Lifecycle transitions require authorized caller context.
- Rejected transitions must be auditable.
- Maintenance mode must preserve access-control boundaries.

## Metrics

- Transition success rate.
- Average startup time.
- Shutdown completion time.
- Invalid transition attempts.

## Tests

- State machine transition tests.
- Unauthorized transition tests.
- Degraded mode tests.
- Recovery transition tests.

## Acceptance Criteria

- All lifecycle states are defined.
- Invalid transitions are rejected and logged.
- Subsystem readiness gates core availability.

## Traceability

- MAL-V00-B010
- MAL-V00-B014
- MAL-V02-B009

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.04 - Runtime Manager

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-004

## Purpose

Define the Runtime Manager that supervises executable runtimes inside the Enterprise Core.

## Scope

Covers runtime creation, supervision, registration, isolation expectations, restart policy, and lifecycle integration.

## Responsibilities

- Start and stop approved runtimes.
- Coordinate runtime lifecycle with kernel state.
- Enforce runtime registration rules.
- Surface runtime health and failure events.

## Inputs

- Runtime definitions.
- Kernel lifecycle state.
- Configuration values.
- Runtime registry entries.

## Outputs

- Runtime instances.
- Runtime status records.
- Runtime failure events.
- Restart decisions.

## Interfaces

- Runtime registry API.
- Lifecycle manager API.
- Health manager API.
- Event bus topics.

## Events

- RuntimeStartRequested
- RuntimeStarted

- RuntimeFailed
- RuntimeRestarted
- RuntimeStopped

## Storage

- Runtime instance records.
- Runtime status journal.
- Restart history.

## Security

- Runtime actions require explicit permission.
- Runtime isolation boundaries must be documented.
- Runtime errors must not leak sensitive payloads.

## Metrics

- Runtime uptime.
- Restart count.
- Runtime startup latency.
- Failure recovery time.

## Tests

- Runtime startup tests.
- Restart policy tests.
- Unauthorized runtime action tests.
- Failure event tests.

## Acceptance Criteria

- Runtime manager supervises approved runtimes only.
- Runtime failures are observable.
- Restart policy is bounded and documented.

## Traceability

- MAL-V00-B012
- MAL-V02-B009
- MAL-V02-B010

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.05 - Runtime Registry

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-005

## Purpose

Define the Runtime Registry as the canonical inventory of runtimes available to the Enterprise Core.

## Scope

Covers runtime metadata, allowed states, versioning, capability declarations, ownership, and compatibility rules.

## Responsibilities

- Store approved runtime definitions.
- Expose runtime lookup and compatibility metadata.
- Track runtime lifecycle status.
- Prevent unregistered runtime execution.

## Inputs

- Runtime manifests.
- Provider metadata.
- Compatibility requirements.
- Security classifications.

## Outputs

- Runtime catalog entries.
- Compatibility decisions.
- Runtime availability state.
- Registry change events.

## Interfaces

- Runtime manager lookup API.
- Configuration interface.
- Security policy interface.
- Manifest validation interface.

## Events

- RuntimeRegistered
- RuntimeUpdated

- RuntimeDeprecated
- RuntimeRejected

## Storage

- Runtime manifest store.
- Registry version history.
- Compatibility metadata records.

## Security

- Registry writes require administrative authority.
- Runtime manifests must be validated.
- Deprecated runtimes must remain traceable.

## Metrics

- Registry lookup latency.
- Manifest validation failure count.
- Deprecated runtime usage.
- Runtime catalog coverage.

## Tests

- Manifest validation tests.
- Registry permission tests.
- Compatibility lookup tests.
- Deprecation tests.

## Acceptance Criteria

- Every runtime has owner, version, status, and security classification.
- Runtime manager cannot start unknown runtimes.
- Registry changes are auditable.

## Traceability

- MAL-V00-B009
- MAL-V02-B009
- MAL-V02-B010

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.06 - Event Bus

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-006

## Purpose

Define the core Event Bus used by Enterprise Core subsystems to publish and consume governed events.

## Scope

Covers event topics, envelopes, delivery expectations, ordering assumptions, auditing, and failure handling.

## Responsibilities

- Provide core event transport semantics.
- Standardize event envelope metadata.
- Route subsystem events without hidden coupling.
- Support audit and observability requirements.

## Inputs

- Event envelopes.
- Publisher identity.
- Subscriber registrations.
- Delivery configuration.

## Outputs

- Delivered events.
- Dead-letter records.
- Event metrics.
- Audit references.

## Interfaces

- Publisher API.
- Subscriber API.
- Dead-letter interface.
- Observability interface.

## Events

- EventPublished
- EventDelivered



- EventDeliveryFailed
- EventDeadLettered

## Storage

- Event journal metadata.
- Dead-letter queue records.
- Subscriber registry.

## Security

- Events must include caller and tenant context when relevant.
- Sensitive payloads require classification.
- Subscribers must be authorized for protected topics.

## Metrics

- Event throughput.
- Delivery latency.
- Dead-letter count.
- Unauthorized subscription attempts.

## Tests

- Envelope validation tests.
- Subscription authorization tests.
- Delivery failure tests.
- Dead-letter handling tests.

## Acceptance Criteria

- Core events use a consistent envelope.
- Failed delivery is observable.
- Protected topics enforce authorization.

## Traceability

- MAL-V00-B010
- MAL-V00-B011
- MAL-V02-B011

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.07 - Service Container

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-007

## Purpose

Define the Service Container that binds and resolves Enterprise Core services.

## Scope

Covers service registration, dependency resolution, scope, lifecycle hooks, and replacement rules.

## Responsibilities

- Register core services with explicit contracts.
- Resolve dependencies safely.
- Support service lifecycle hooks.
- Prevent circular or hidden dependencies.

## Inputs

- Service definitions.
- Dependency declarations.
- Kernel boot context.
- Configuration values.

## Outputs

- Service instances.
- Resolution diagnostics.
- Dependency graph.
- Service lifecycle events.

## Interfaces

- Kernel service lookup.
- Configuration provider.
- Lifecycle hooks.
- Health checks.

## Events

- ServiceRegistered
- ServiceResolved

- ServiceResolutionFailed
- ServiceDisposed

### Storage

- Service registry.
- Dependency graph snapshot.
- Resolution logs.

### Security

- Service resolution must respect caller authority where needed.
- Sensitive service configuration must not be logged.
- Privileged services must be explicitly marked.

### Metrics

- Resolution latency.
- Circular dependency count.
- Service startup failures.
- Privileged service access attempts.

### Tests

- Dependency graph tests.
- Circular dependency tests.
- Privileged resolution tests.
- Service disposal tests.

### Acceptance Criteria

- Core services have explicit interfaces.
- Invalid dependency graphs fail fast.
- Service lifecycle is observable.

### Traceability

- MAL-V00-B012
- MAL-V02-B003
- MAL-V02-B018

### Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.08 - Configuration

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-008

## Purpose

Define configuration management for Enterprise Core services and runtimes.

## Scope

Covers configuration sources, validation, environment scope, defaults, secrets separation, and change events.

## Responsibilities

- Load configuration from approved sources.
- Validate required settings before startup.
- Separate secrets from ordinary configuration.
- Expose configuration changes to dependent subsystems.

## Inputs

- Environment values.
- Configuration files.
- Runtime manifests.
- Policy defaults.

## Outputs

- Validated configuration objects.
- Configuration error reports.
- Configuration change events.
- Effective configuration snapshots.

## Interfaces

- Kernel configuration API.
- Runtime manager configuration interface.
- Security secret provider interface.
- Observability reporting.

## Events

- ConfigurationLoaded
- ConfigurationValidated

- ConfigurationRejected
- ConfigurationChanged

## Storage

- Effective configuration snapshot.
- Configuration validation report.
- Non-secret configuration history.

## Security

- Secrets must never be stored in plaintext Markdown or release artifacts.
- Configuration reads must respect environment boundaries.
- Sensitive values must be redacted from logs.

## Metrics

- Configuration validation failure count.
- Startup blocked by configuration.
- Configuration reload time.
- Secret access audit count.

## Tests

- Missing configuration tests.
- Invalid type tests.
- Secret redaction tests.
- Environment override tests.

## Acceptance Criteria

- Core cannot start with invalid required configuration.
- Secrets are handled through a separate provider.
- Effective configuration can be inspected safely.

## Traceability

- MAL-V00-B011
- MAL-V02-B018
- MAL-V03-B03-01-002

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.09 - Identity Access

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-009

## Purpose

Define identity and access primitives required by the Enterprise Core.

## Scope

Covers users, service identities, tenant context, roles, permissions, authorization checks, and audit decisions.

## Responsibilities

- Represent human and service identities.
- Resolve tenant and organization context.
- Authorize protected core actions.
- Emit audit evidence for access decisions.

## Inputs

- Authentication assertions.
- Tenant context.
- Role assignments.
- Permission policies.
- Service identity manifests.

## Outputs

- Authorization decisions.
- Identity context objects.
- Access audit events.
- Denied action records.

## Interfaces

- Kernel authorization guard.
- Service container privileged resolution.
- Event bus topic authorization.
- Audit event interface.

## Events

- IdentityContextResolved

- AuthorizationGranted
- AuthorizationDenied
- RoleBindingChanged

## Storage

- Identity records references.
- Role binding records.
- Permission policy versions.
- Access audit log.

## Security

- Use least privilege for humans and services.
- Deny by default for protected actions.
- Audit denied and privileged actions.

## Metrics

- Authorization latency.
- Denied action count.
- Privileged action count.
- Role binding change rate.

## Tests

- Permission matrix tests.
- Tenant isolation tests.
- Service identity tests.
- Denied action audit tests.

## Acceptance Criteria

- Protected core actions require authorization.
- Tenant context is explicit.
- Access decisions are auditable.

## Traceability

- MAL-V00-B005
- MAL-V00-B011
- MAL-V02-B003

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.10 - State Manager

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-010

## Purpose

Define the State Manager responsible for controlled state transitions inside the Enterprise Core.

## Scope

Covers state ownership, transition validation, persistence expectations, consistency, and recovery.

## Responsibilities

- Own canonical core state transitions.
- Validate state changes against lifecycle rules.
- Expose state snapshots to authorized services.
- Support recovery after failure.

## Inputs

- State transition commands.
- Current state snapshots.
- Lifecycle constraints.
- Caller context.

## Outputs

- Accepted state changes.
- Rejected transition records.
- State snapshots.
- Recovery checkpoints.

## Interfaces

- Lifecycle manager interface.
- Storage interface.
- Event bus state events.
- Health manager state checks.

## Events

- StateChangeRequested
- StateChanged



- StateChangeRejected
- StateRecovered

## Storage

- State snapshot store.
- Transition journal.
- Recovery checkpoints.

## Security

- State mutations require authorized callers.
- Sensitive state values must be classified.
- Recovery data must preserve tenant boundaries.

## Metrics

- Transition success rate.
- Rejected transition count.
- State recovery time.
- Snapshot write latency.

## Tests

- State transition tests.
- Invalid mutation tests.
- Recovery tests.
- Concurrent state update tests.

## Acceptance Criteria

- State transitions are validated and observable.
- Invalid mutations are rejected.
- Recovery restores a consistent state.

## Traceability

- MAL-V00-B010
- MAL-V02-B009
- MAL-V02-B011

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.11 - Resource Manager

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-011

## Purpose

Define management of core resources such as compute slots, quotas, handles, and shared platform limits.

## Scope

Covers resource allocation, quota enforcement, release, contention handling, and observability.

## Responsibilities

- Track resource availability.
- Allocate resources to approved runtimes and services.
- Enforce quotas and limits.
- Release resources reliably.

## Inputs

- Resource requests.
- Quota policies.
- Runtime identity.
- Current utilization signals.

## Outputs

- Resource grants.
- Resource denial events.
- Utilization metrics.
- Quota breach records.

## Interfaces

- Runtime manager allocation API.
- Configuration quota provider.
- Observability metrics interface.
- Event bus resource topics.

## Events

- ResourceRequested
- ResourceGranted

- ResourceDenied
- ResourceReleased
- QuotaExceeded

## Storage

- Resource allocation ledger.
- Quota policy records.
- Utilization snapshots.

## Security

- Resource grants must respect tenant and service permissions.
- Quota bypass requires explicit privileged authority.
- Resource metrics must avoid exposing sensitive workload payloads.

## Metrics

- Resource utilization.
- Quota denial count.
- Allocation latency.
- Resource leak count.

## Tests

- Quota enforcement tests.
- Resource release tests.
- Unauthorized allocation tests.
- Contention handling tests.

## Acceptance Criteria

- Resources are allocated only through the manager.
- Quota breaches are visible.
- Released resources are reclaimed.

## Traceability

- MAL-V00-B005
- MAL-V02-B018
- MAL-V03-B03-01-004

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.12 - Object Manager

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-012

## Purpose

Define the Object Manager for canonical enterprise objects managed by the core.

## Scope

Covers object identity, ownership, lifecycle, relationships, validation, and event emission.

## Responsibilities

- Assign stable object identity.
- Validate object lifecycle transitions.
- Track ownership and relationship metadata.
- Emit object change events.

## Inputs

- Object creation commands.
- Object update commands.
- Ownership context.
- Relationship definitions.

## Outputs

- Canonical object records.
- Object lifecycle events.
- Validation errors.
- Relationship indexes.

## Interfaces

- State manager interface.
- Storage interface.
- Identity access interface.
- Event bus object topics.

## Events

- ObjectCreated
- ObjectUpdated

- ObjectArchived
- ObjectValidationFailed

## Storage

- Object store.
- Object relationship index.
- Object lifecycle journal.

## Security

- Object access follows identity and tenant rules.
- Protected object changes are audited.
- Archived objects retain governance metadata.

## Metrics

- Object creation rate.
- Validation failure count.
- Object lookup latency.
- Archived object count.

## Tests

- Object lifecycle tests.
- Ownership authorization tests.
- Relationship integrity tests.
- Archive and recovery tests.

## Acceptance Criteria

- Objects have stable IDs and owners.
- Invalid object changes are rejected.
- Object changes emit traceable events.

## Traceability

- MAL-V00-B005
- MAL-V01-B004
- MAL-V02-B003

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.13 - Health Manager

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-013

## Purpose

Define health management for Enterprise Core subsystems, services, and runtimes.

## Scope

Covers readiness, liveness, degradation, dependency checks, incident signals, and health reporting.

## Responsibilities

- Collect health checks from core subsystems.
- Determine readiness and liveness state.
- Surface degradation and incidents.
- Feed lifecycle decisions.

## Inputs

- Subsystem health reports.
- Runtime status.
- Dependency check results.
- Configuration thresholds.

## Outputs

- Health status records.
- Readiness decisions.
- Incident signals.
- Health dashboards data.

## Interfaces

- Kernel readiness API.
- Runtime manager health API.
- Observability metrics interface.
- Event bus health topics.

## Events

- HealthCheckReported
- CoreReady

- CoreDegraded
- CoreUnhealthy
- IncidentSignalRaised

## Storage

- Health snapshots.
- Dependency check history.
- Incident signal records.

## Security

- Health checks must not expose secrets.
- Administrative health detail requires authorization.
- Incident signals must preserve tenant boundaries.

## Metrics

- Readiness pass rate.
- Health check latency.
- Degradation duration.
- Incident signal count.

## Tests

- Readiness tests.
- Dependency failure tests.
- Health authorization tests.
- Degradation event tests.

## Acceptance Criteria

- Core readiness reflects subsystem health.
- Degradation is observable.
- Health data is safe to expose at each authorization level.

## Traceability

- MAL-V00-B010
- MAL-V02-B018
- MAL-V03-B03-01-003

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.14 - Core Events

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-014

## Purpose

Define the canonical event taxonomy for Enterprise Core.

## Scope

Covers event naming, envelopes, categories, severity, correlation, causation, and retention expectations.

## Responsibilities

- Standardize core event names.
- Define required event envelope fields.
- Classify event severity and category.
- Support traceability across subsystem boundaries.

## Inputs

- Subsystem event proposals.
- Event bus envelope requirements.
- Audit and observability needs.
- Security classification rules.

## Outputs

- Core event catalog.
- Event envelope standard.
- Retention guidance.
- Correlation and causation rules.

## Interfaces

- Event bus publisher interface.
- Audit log interface.
- Observability trace interface.
- Storage retention interface.

## Events

- CoreEventDefined
- CoreEventDeprecated



- CoreEventEmitted
- CoreEventRejected

## Storage

- Event catalog.
- Event schema history.
- Event retention metadata.

## Security

- Event payloads must be classified.
- Protected event streams require authorization.
- Event retention must respect data policy.

## Metrics

- Event schema coverage.
- Malformed event count.
- Correlation coverage.
- Protected stream access denial count.

## Tests

- Event schema validation tests.
- Correlation propagation tests.
- Protected event access tests.
- Deprecated event tests.

## Acceptance Criteria

- Core events follow naming and envelope rules.
- Events can be correlated across subsystems.
- Sensitive events are protected.

## Traceability

- MAL-V00-B009
- MAL-V00-B011
- MAL-V02-B011

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.15 - Core Storage

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-015

## Purpose

Define core storage responsibilities for Enterprise Core records and operational metadata.

## Scope

Covers logical stores, record ownership, retention, backup expectations, migration posture, and consistency.

## Responsibilities

- Define logical stores for core records.
- Support persistence for state, registry, audit, health, and configuration metadata.
- Declare retention and backup expectations.
- Avoid binding blueprint to one database product.

## Inputs

- Core record schemas.
- Retention policy inputs.
- Backup requirements.
- Storage classification.

## Outputs

- Logical storage map.
- Persistence requirements.
- Retention rules.
- Migration guidance.

## Interfaces

- State manager storage API.
- Object manager storage API.
- Audit store interface.
- Backup and recovery interface.

## Events

- StorageWriteRequested
- StorageWriteCommitted

- StorageReadDenied
- StorageRecoveryStarted

## Storage

- State snapshots.
- Runtime registry records.
- Object records.
- Audit references.
- Health snapshots.

## Security

- Encrypt sensitive storage where required.
- Authorize protected reads and writes.
- Keep tenant data isolated.

## Metrics

- Storage write latency.
- Read denial count.
- Backup success rate.
- Migration failure count.

## Tests

- Persistence tests.
- Tenant isolation storage tests.
- Backup restore tests.
- Migration compatibility tests.

## Acceptance Criteria

- Core storage map is documented.
- Protected records enforce access rules.
- Recovery expectations are explicit.

## Traceability

- MAL-V00-B011
- MAL-V02-B018
- MAL-V03-B03-01-010

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.16 - Core Security

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-016

## Purpose

Define security controls that every Enterprise Core subsystem must respect.

## Scope

Covers least privilege, tenant isolation, secrets, audit, policy enforcement, secure defaults, and threat-aware design.

## Responsibilities

- Set mandatory core security controls.
- Define protected action handling.
- Separate secrets from configuration.
- Ensure auditability of privileged operations.

## Inputs

- Security principles.
- Identity and access decisions.
- Configuration and secret provider data.
- Threat and risk assumptions.

## Outputs

- Core security control baseline.
- Protected action catalog.
- Audit requirements.
- Security acceptance checklist.

## Interfaces

- Identity access interface.
- Configuration secret provider.
- Audit event bus topics.
- Storage security hooks.

## Events

- ProtectedActionRequested
- ProtectedActionDenied

- SecretAccessed
- SecurityPolicyChanged

## Storage

- Protected action records.
- Security policy versions.
- Audit event store.
- Secret access metadata.

## Security

- Deny by default.
- Never store secrets in release artifacts.
- Audit privileged and denied actions.
- Keep tenant isolation explicit.

## Metrics

- Denied protected action count.
- Secret access count.
- Policy change count.
- Security test pass rate.

## Tests

- Least privilege tests.
- Secret redaction tests.
- Tenant isolation tests.
- Protected action audit tests.

## Acceptance Criteria

- Every core subsystem states security behavior.
- Protected actions are known and auditable.
- Secrets are separated from ordinary configuration.

## Traceability

- MAL-V00-B011
- MAL-V02-B016
- MAL-V02-B018

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.17 - Core Observability

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-017

## Purpose

Define observability for Enterprise Core operations, health, performance, security, and user-impacting failures.

## Scope

Covers metrics, logs, traces, audit correlation, dashboards, alerts, and incident evidence.

## Responsibilities

- Standardize core metrics and logs.
- Propagate correlation IDs.
- Expose health and performance indicators.
- Support incident analysis.

## Inputs

- Subsystem metrics.
- Event bus correlation data.
- Health snapshots.
- Runtime status.
- Security audit metadata.

## Outputs

- Metrics streams.
- Structured logs.
- Trace spans.
- Operational dashboards.
- Alert signals.

## Interfaces

- Health manager metrics API.
- Event bus observability hooks.
- Runtime manager telemetry.
- Storage telemetry interface.

## Events

- MetricEmitted
- TraceStarted
- TraceCompleted
- AlertRaised
- ObservationDropped

## Storage

- Metrics time series.
- Structured log store.
- Trace metadata.
- Alert history.

## Security

- Logs and traces must redact secrets.
- Sensitive observations require access controls.
- Audit evidence must remain tamper-resistant.

## Metrics

- Metric coverage.
- Log error rate.
- Trace correlation coverage.
- Alert precision.

## Tests

- Redaction tests.
- Metric emission tests.
- Trace propagation tests.
- Alert threshold tests.

## Acceptance Criteria

- Critical core paths emit useful telemetry.
- Sensitive data is not exposed in observations.
- Incidents can be reconstructed from approved signals.

## Traceability

- MAL-V00-B010
- MAL-V02-B018
- MAL-V03-B03-01-013

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |



# Book 03.01.18 - Core Testing

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-018

## Purpose

Define the testing blueprint for Enterprise Core subsystems.

## Scope

Covers unit, integration, system, security, performance, recovery, and acceptance testing for the core.

## Responsibilities

- Define test layers for core services.
- Map subsystem risks to test types.
- Establish release gating expectations.
- Prepare inputs for Volume 09 Testing Guide.

## Inputs

- Subsystem blueprints.
- Acceptance criteria.
- Security controls.
- Runtime and lifecycle states.

## Outputs

- Core test matrix.
- Acceptance test scenarios.
- Security test requirements.
- Performance and recovery test targets.

## Interfaces

- CI test runner interface.
- Runtime test harness.
- Security test hooks.
- Observability verification.

## Events

- TestPlanCreated
- CoreTestStarted

- CoreTestFailed
- CoreAcceptancePassed

## Storage

- Test result records.
- Coverage reports.
- Failure evidence.
- Acceptance signoff records.

## Security

- Security tests must avoid real secrets.
- Test fixtures must isolate tenant data.
- Privileged test paths must be explicit.

## Metrics

- Test pass rate.
- Coverage by subsystem.
- Flaky test rate.
- Mean time to diagnose failures.

## Tests

- Unit tests for service logic.
- Integration tests for subsystem contracts.
- Security tests for protected actions.
- Recovery tests for lifecycle and storage.

## Acceptance Criteria

- Every core subsystem has named test expectations.
- Acceptance tests map to blueprint criteria.
- Build verification includes PDF and ZIP checks.

## Traceability

- MAL-V00-B010
- MAL-V00-B012
- MAL-V02-B003

## Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.19 - Core Acceptance

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-019

## Purpose

Define acceptance gates for the Enterprise Core blueprint and future implementation.

## Scope

Covers blueprint completeness, subsystem contracts, security readiness, runtime readiness, test readiness, and release evidence.

## Responsibilities

- Define acceptance gates for core design.
- Connect subsystem criteria into a single review model.
- Clarify what is required before implementation can be considered ready.

## Inputs

- Subsystem acceptance criteria.
- Traceability records.
- Security baseline.
- Testing blueprint.
- Release artifact evidence.

## Outputs

- Core acceptance checklist.
- Readiness decision model.
- Release evidence requirements.
- Gap register.

## Interfaces

- Blueprint index.
- Testing guide inputs.
- Manifest and release notes.
- Future specification references.

## Events

- CoreAcceptanceReviewStarted
- CoreAcceptanceGapFound

- CoreAcceptanceApproved
- CoreAcceptanceRejected

### Storage

- Acceptance checklist records.
- Gap register.
- Review notes.
- Release evidence links.

### Security

- Acceptance evidence must not contain secrets.
- Security gaps block approval.
- Review authority must be explicit.

### Metrics

- Acceptance checklist completion.
- Open critical gap count.
- Traceability coverage.
- Security gate pass rate.

### Tests

- Document section validation.
- Traceability validation.
- Security checklist validation.
- Release artifact validation.

### Acceptance Criteria

- All Enterprise Core documents exist.
- Required sections are present.
- PDF and ZIP artifacts include the core blueprint.
- Critical security gaps are recorded or resolved.

### Traceability

- MAL-V00-B009
- MAL-V00-B014
- MAL-V02-B003

### Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |

# Book 03.01.20 - Core Roadmap

Version: v3 draft

Status: Draft

Document ID: MAL-V03-B03-01-020

## Purpose

Define the implementation-facing roadmap for moving Enterprise Core from blueprint to specification and code.

## Scope

Covers sequencing, specification priorities, implementation slices, dependencies, migration concerns, and future expansion.

## Responsibilities

- Translate blueprint into build-ready phases.
- Identify first specifications to write.
- Sequence implementation around kernel, identity, lifecycle, runtime, storage, security, and observability.
- Preserve alignment with Volume 02 roadmap.

## Inputs

- Enterprise Core blueprint documents.
- Master Roadmap phase 01.
- Engineering standards backlog.
- Testing and operations guide needs.

## Outputs

- Core specification backlog.
- Implementation sequence proposal.
- Dependency map.
- Risk and migration notes.

## Interfaces

- Volume 06 specification queue.
- Volume 05 engineering standards.
- Volume 09 testing guide.
- System repository implementation issues.

## Events

- CoreRoadmapUpdated
- CoreSpecificationQueued

- CoreImplementationGateOpened
- CoreImplementationGateBlocked

Storage

- Roadmap decision records.
- Specification backlog.
- Implementation gate evidence.

Security

- Implementation must not begin without security and identity specifications.
- Roadmap records must avoid secrets.
- Code repository references must remain traceable to document IDs.

Metrics

- Specification readiness count.
- Blocked dependency count.
- Implementation gate completion.
- Roadmap change frequency.

Tests

- Roadmap consistency tests.
- Dependency completeness review.
- Traceability review.
- Implementation readiness checklist.

Acceptance Criteria

- The roadmap identifies specification order.
- The roadmap blocks premature autonomy or marketplace work before core foundations.
- Implementation gates are traceable to blueprint acceptance.

Traceability

- MAL-V00-B014
- MAL-V02-B003
- MAL-V03-B03-01-019

Version History

| Version  | Date       | Status | Notes                                                                         |
|----------|------------|--------|-------------------------------------------------------------------------------|
| v3 draft | 2026-06-29 | Draft  | Initial Enterprise Core blueprint content for Mariam Architecture Library v3. |